

Addis Ababa Ride Demand Forecasting

End-to-end project documentation

Qiyas AI Hackathon · hourly demand per zone, 1–14 November 2025

Team	TEAM ALPHA
Final model	LightGBM (tree-based model, set up for counts), average of 3 training runs
Average miss on 18–31 Oct (MAE)	5.85 trips per zone-hour ($\approx$ 4.5 drivers)
RMSE on 18–31 Oct (weighs large misses more)	8.47 trips per zone-hour
RMSE of the simple rule (usual value for zone, weekday, hour)	11.30
RMSE averaged over 4 two-week test periods	9.21 ± 0.71
Document generated	06 October 2026

Contents

1. Project overview	3
1.1 Key terms	3
1.2 The input data	3
1.3 Approach in one paragraph	4
2. Data cleaning & integration (Deliverable A)	5
2.1 Timestamp parsing and the clock problem	5
2.2 Zone and event-type standardisation	5
2.3 Cleaning log (A1)	5
2.4 Join map (A3)	7
2.5 Join audit, proof and integrity checks (A4, A5, A7)	8
2.6 Master tables and features (A6, A8)	8
3. Analysis findings (Deliverable B)	9
3.1 Zones and time patterns	9
3.2 Weather	10
3.3 Events and holidays	11
3.4 Data quality and operational columns	12
4. Visualisation pack (Deliverable C)	14
5. Modelling & evaluation (Deliverable D)	15
5.1 Rules followed	15
5.2 Baselines and model comparison (D1, D2)	15
5.3 Tuning, rolling-origin validation and ablation (D6, D3, D5)	16
5.4 Error analysis and response (D7, D8)	16
5.5 Optimisation round (D10)	17
5.6 What the error means in practice (D9)	17
5.7 Feature importance	18
5.8 Final model and submission	19
6. Forecast demo app (Deliverable E)	20
7. Reproducibility & project structure (Deliverable G)	21
7.1 Environment	21
7.2 Run order	21
7.3 Reusable pipeline (src/)	21
7.4 Folder layout	21
8. Limitations and next steps	22

1. Project overview

A ride-hailing operator in Addis Ababa needs to know **how many trips will be requested in each of 12 zones, every hour, for the next two weeks**, so it can position drivers and plan incentives. We are given ten months of hourly trip history (1 Jan – 31 Oct 2025), an hourly weather table (observations plus a forecast for November) and an events calendar (football matches, concerts, conferences, road closures, holidays...). The task is to forecast 4,032 zone-hours (12 zones × 14 days × 24 h) for 1–14 November 2025.

The data are deliberately messy. Before any modelling, the three tables have to be cleaned and joined on a common clock and common zone names. The project is organised in the seven hackathon deliverables:

ID	Deliverable	Where it lives
A	Cleaning & integration (log, join map, audit, proof, master tables)	notebooks/01, reports/A_cleaning_and_integration.html, data/processed/
B	Analysis report (zones, weather, events, data quality)	notebooks/02, reports/B_analysis_report.html
C	Visualisation pack (12 figures + captions)	notebooks/03, figures/
D	Modelling & evaluation	notebooks/04, reports/D_model_evaluation.html, models/
E	Forecast demo app	app/ (Streamlit)
F	5-slide presentation	presentation/
G	Structure & reproducibility	README.md, requirements.txt, src/
—	Submission file	submission/team_alpha_submission.csv

1.1 Key terms

Term	Meaning in this project
Zone-hour	one zone in one hour, e.g. Bole on 5 Nov at 18:00. Forecasts are made per zone-hour.
EAT / UTC	EAT is local Addis Ababa time. UTC is world time, 3 hours behind EAT.
MAE	mean absolute error: the average miss, in trips per zone-hour. Lower is better.
RMSE	root mean squared error: like MAE but large misses count more. The hackathon's main score.
Baseline / seasonal-naive	a simple rule to beat: predict the usual value for the same zone, weekday and hour.
Lag	the trip count at the same zone and hour some time earlier, e.g. a 336 h lag = 14 days earlier.
Validation	testing the model on weeks it did not see during training (here, late October).
Rolling-origin folds	repeating that test on several two-week periods (6 Sep, 20 Sep, 4 Oct, 18 Oct).
Leakage	using information that would not be known when the forecast is made, which gives falsely good scores.
Ablation	retraining with groups of inputs removed to measure how much each group helps.
Feature	one input column given to the model, e.g. rain_mm or hour.

1.2 The input data

File	Content	Main problems found
ride_demand_train.csv	85,460 rows: zone, pickup_hour, trips, avg_fare_birr, avg_wait_min, active_drivers	3 timestamp formats, 55 zone spellings, duplicates, –1 sentinels, impossible spikes, outage, late-launching zone
ride_demand_test.csv	4,032 rows: row_id, zone, pickup_hour	same timestamp / zone issues
weather_hourly.csv	7,538 rows: temp, rain, humidity, wind, observed/forecast	on UTC (not local time), –9999 rain codes, a July block in °F, duplicate and missing hours
events_calendar.csv	165 events: type, venue, zone, start, end, attendance, status	20 type spellings, multi-zone / city-wide zones, 3 date formats, missing or reversed ends, free-text attendance, cancelled events
submission_template.csv	row order for the submission	—

1.3 Approach in one paragraph

Every timestamp is parsed with an explicit, regex-detected format and converted to East Africa Time (EAT, UTC+3). Zones are mapped to 12 canonical names. Trips, weather and events are cleaned with documented rules, then joined: weather many-to-one on the hour, events as an **interval join** (2 h before start to 2 h after end, per affected zone). Analysis quantifies zone profiles, growth, rain and event effects. A LightGBM model uses only information that exists at forecast time — calendar, weather forecast, scheduled events and trip history at least 14 days old — and is validated chronologically. A Streamlit app turns the model into an operations tool, and a `src/` package makes the whole pipeline reusable from the command line.

2. Data cleaning & integration (Deliverable A)

2.1 Timestamp parsing and the clock problem

Each table mixes several timestamp formats (ISO with +03:00, ISO with Z, yyyy-mm-dd hh:mm, dd/mm/yyyy hh:mm, and Feb 06, 2025 08:00 PM). Letting pandas guess silently swaps day and month (05/11 read as 11 May). Instead, each format is detected with a regular expression and parsed explicitly. Day-first was proven: in the dd/mm strings the first number exceeds 12 in thousands of rows while the second never does.

Time-zone proof. The trip file is on local time (ISO rows carry +03:00, and all formats fall on one clean hourly grid). The weather file is on **UTC**: its ISO strings end in Z, it starts at 30 Dec 21:00 Z = 1 Jan 00:00 EAT, and on the raw clock temperature peaks at 12:00 — solar noon — instead of the usual mid-afternoon. After shifting +3 h the peak lands at 15:00 EAT and the rain–demand correlation jumps from 0.06 to 0.42. Weather was therefore shifted +3 h; trips and events were left unchanged.

fig06 – Weather timestamps are UTC: shifting +3 h aligns temperature and rain with demand

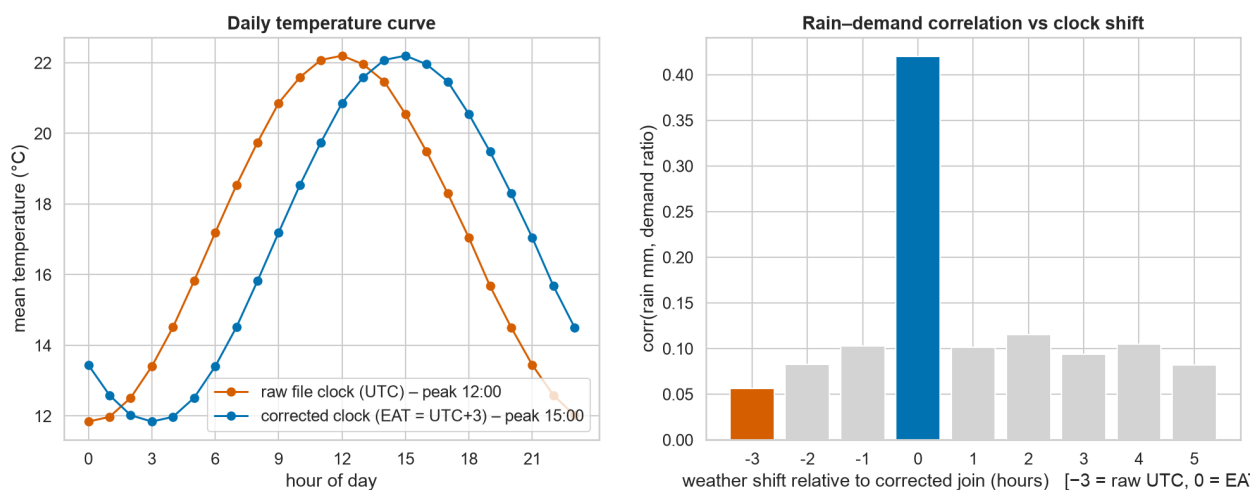


fig06_weather_timezone_check.png — Daily temperature curve on the raw vs corrected clock, and rain–demand correlation for each clock shift. So what: the weather file is in UTC; the +3 h correction puts the temperature peak at 15:00 and raises the rain–demand correlation from 0.06 to 0.42.

2.2 Zone and event-type standardisation

The raw files contain 55 zone spellings (case, spacing, aliases such as *Piazza*, *Mercato*, *Bole Rd*, *Kolfe Keranio*, text in brackets such as *(Kirkos)*). A normaliser lower-cases, collapses spaces, strips bracketed text and applies an alias map to the 12 canonical zones: Arat Kilo, Ayat, Bole, CMC, Gerji, Kazanchis, Kolfe, Lideta, Megenagna, Merkato, Piassa, Sarbet. Events naming several zones (*Lideta & KAZANCHIS*) are split, and city-wide events are expanded to all 12. Twenty event-type spellings were normalised to 8 labels.

2.3 Cleaning log (A1)

Every issue found, how many rows it affected and the fix applied:

Table	Column	Issue	Rows	%	Fix
ride demand train	pickup_hour	mixed timestamp formats (3)	38,574	45.1	regex-detected explicit formats; +03:00 converted to EAT
ride demand train	zone	inconsistent zone spelling (55 variants)	32,391	37.9	case/space normalization + alias map to 12 labels
ride demand train	zone, pickup_hour	duplicate zone-hour rows	846	1.0	kept first occurrence

Table	Column	Issue	Rows	%	Fix
ride demand train	trips	sentinel -1 (no reading)	675	0.8	set to NaN, row excluded from training target
ride demand train	trips	missing target	837	1.0	row excluded from training target (kept in grid for lags)
ride demand train	trips	data-entry spikes (trips > 4x active drivers)	123	0.1	set to NaN
ride demand train	avg_wait_min	sentinel -1	1,692	2.0	set to NaN
ride demand train	avg_fare_birr	missing values	1,269	1.5	left NaN (not a model input)
ride demand train	(rows)	platform outage – whole city missing	42	0.6	not imputed; hours excluded from training
ride demand train	(rows)	late-launching zone (Ayat) – no rows before launch	1,752	2.0	zone-hours before launch excluded
weather hourly	timestamp	UTC clock + 2 formats	7,538	100.0	parsed explicitly, converted UTC -> EAT (+3h)
weather hourly	rain_mm	sentinel -9999	76	1.0	set to NaN, later filled
weather hourly	temp_c	values in Fahrenheit for part of July	353	4.7	converted (F-32)*5/9
weather hourly	timestamp	duplicate hours with conflicting values	75	1.0	averaged numeric values per hour
weather hourly	timestamp	missing hours (gaps)	192	2.5	reindexed to full hourly grid; interpolated (≤ 6 h), rain filled 0
weather hourly	temp_c, humidity_pct	missing values	193	2.6	time interpolation
events calendar	event_type	inconsistent spelling (20 variants)	94	57.0	normalized to 8 labels
events calendar	status	case/whitespace variants	28	17.0	lower + strip
events calendar	zone	spelling, extra text, multi-zone and city-wide values	87	52.7	split on '&', strip '(...)', alias map, citywide -> all 12
events calendar	start_datetime, end_datetime	3 timestamp formats incl. 'Feb 06, 2025 08:00 PM'	170	51.5	explicit regex formats
events calendar	end_datetime	missing or earlier than start	5	3.0	replaced by start + median duration of that event type
events calendar	expected_attendance	free text ('approx 34000', '34,756') and blanks	111	67.3	parsed to number; blanks -> 0 in features
events calendar	status	cancelled events	6	3.6	excluded from event features (kept in file for B3.4)

Key judgement calls:

- **Impossible spikes:** trips above 4× the active drivers in that hour (e.g. 640 trips with 56 drivers) were set to missing. Real event peaks keep the usual ≈ 1.3 trips per driver.
- **Platform outage** (12 May 02:00 – 13 May 19:00, 42 h with no rows in any zone) and **Ayat before its launch on 15 Mar** were *excluded*, not filled with zeros — no demand was recorded, and zero would teach the model a false pattern.
- **Weather gaps** of up to 6 h are interpolated in time. The one long gap (19 h on 3 Sep) is filled with the hour-of-day average from history only. Missing rain is set to 0, the most common state.
- **Events:** missing or reversed end times are replaced by start + the median duration of that event type. Cancelled events are kept in the file for analysis (B3.4) but excluded from features.

fig01 – Missing and invalid values in the raw tables, and gaps in the trip history

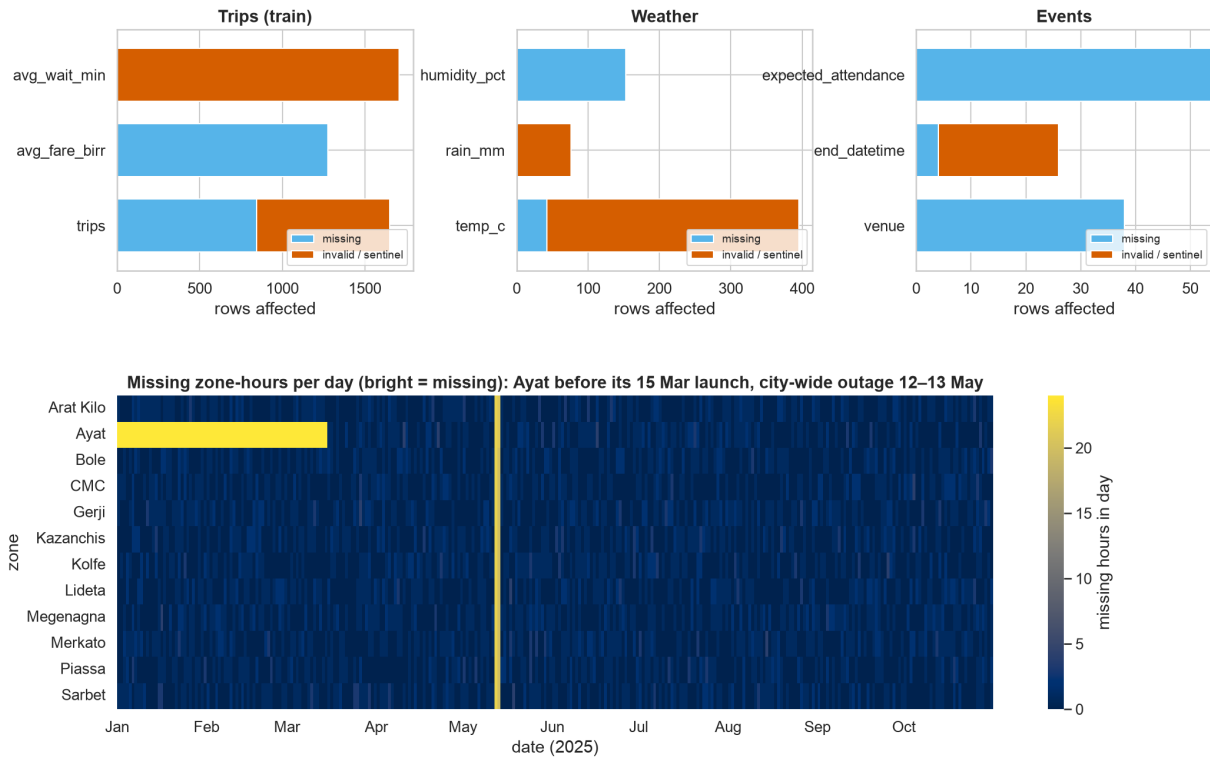


fig01_gaps_and_missingness.png — Missing/invalid values per column in the three raw tables and a zone × day map of missing trip hours. So what: the gaps are structured — Ayat did not exist before 15 Mar and a 42-hour city-wide outage hit 12–13 May — so they are excluded rather than imputed as zero demand.

fig02 – Cleaning removed impossible temperatures, rain sentinels and trip spikes

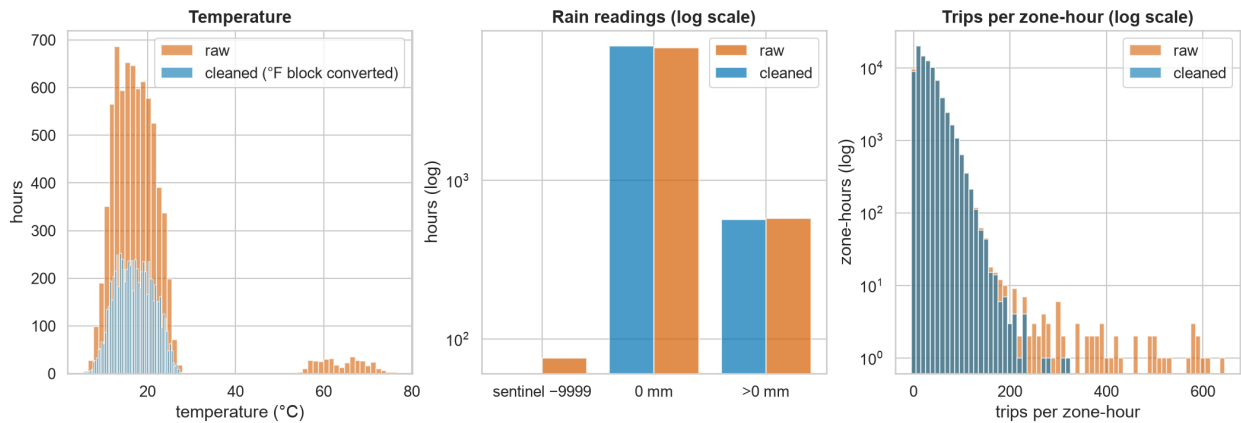


fig02_before_after_cleaning.png — Raw vs cleaned distributions of temperature, rain readings and trips. So what: a July block recorded in °F (60–77), –9999 rain sentinels and trip spikes far beyond driver capacity would each have distorted the model; cleaning removes them without changing the normal range.

2.4 Join map (A3)

```

weather_clean (1 row per hour, EAT)
  | key: datetime          many-to-one, LEFT join
  v
zone-hour grid <---- trips_clean (left table) ----> master_train / master_test
(zone, datetime)   ^
                    | key: zone AND datetime in [start-2h, end+2h]
                    | interval join, aggregated to 1 row per zone-hour, LEFT join
events_clean (exploded to zone x hour)

```

- The **left table** is the set of trip zone-hours (train) or test rows, so joins can never add or drop rows.
- **Weather** is joined many-to-one on the hour (validated m:1).
- **Events** are exploded to every hour from 2 h before start to 2 h after end in each affected zone, labelled pre / during / post. Overlapping events are aggregated (any-of-type flags, max attendance), which makes the join many-to-one again.
- **Public holidays and school breaks** become whole-day, city-wide calendar flags.

2.5 Join audit, proof and integrity checks (A4, A5, A7)

The audit shows a 100% weather match rate for train and test (the few imputed hours are flagged), and counts how many events matched at least one zone-hour. The proof section prints, for a rainy hour, a football window and a public holiday, the exact weather and event rows that were attached, plus the resulting feature values. Thirteen automated integrity checks must all pass before anything is written:

- one row per zone-hour in train and test
- only the 12 canonical zones
- train timestamps in 1 Jan – 31 Oct and test in 1 – 14 Nov (EAT)
- test has 4,032 rows in template order
- no negative or sentinel trips or weather values
- no missing weather after the join
- joins did not change row counts
- train and test share identical feature columns
- no forecast-time-unavailable feature in test
- weather and event features present (Rule 5)

2.6 Master tables and features (A6, A8)

master_train.csv has **82,979** usable zone-hours and master_test.csv has **4,032** rows in submission order. Both carry identical feature columns, documented in data_dictionary_master.csv:

Group	Features	Why it helps
Calendar	hour, day_of_week, is_weekend, day_of_month, is_payday_window, is_public_holiday, is_school_break, days_since_start, hour_sin/cos	daily and weekly shapes, holidays, growth trend
Weather	temp_c, rain_mm, humidity_pct, wind_kmh, rain_3h_sum, is_raining, rain_class	rain moves walkers and minibus riders to ride-hail
Events	event_in_window, event_attendance, football/concert/conference/road_closure_window, event_pre/during/post	crowds arriving and leaving; road closures suppress demand
History	lag_336h, lag_504h, roll_mean_168h_lag336	recent level per zone-hour (≥ 14 days old)
Excluded	active_drivers, avg_wait_min, avg_fare_birr, lag_1h, lag_24h, month	unknown at forecast time or unseen in training

3. Analysis findings (Deliverable B)

Each question in the brief was answered with a table and a written interpretation. The baseline used to measure effects is the zone's ordinary-hour shape (zone × weekday × hour) times its linear growth trend. Ratios are computed as sum over sum, so small hours don't dominate.

3.1 Zones and time patterns

B1.1. Merkato (12.1%), Bole (11.7%) and Megenagna (11.4%) carry over a third of all 2.36 M trips; Arat Kilo and Ayat are the smallest. Ayat only launched on 15 Mar 2025, so its 4.0% share understates it — compare zones on **mean trips per zone-hour** (Ayat 17.8 vs Merkato 40.5), not totals.

B1.2. Five clear shapes: **business/CBD** (Arat Kilo, Kazanchis, Piassa) and **transport hubs** (Lideta, Megenagna) have twin commuter peaks at 08:00 and 18:00 with the evening one strongest; **residential** zones (Ayat, CMC, Gerji, Kolfe, Sarbet) peak earlier in the morning (07:00, outbound) and at 19:00 (home-bound); **Merkato** is a single broad midday plateau peaking at 13:00 and dead after 20:00; **Bole** (airport/nightlife) has a 06:00 airport peak, an evening peak at 20:00 and the highest night-time floor. All groups are quietest at 00:00–02:00.

B1.3. Business/CBD zones collapse at weekends (ratio 0.47–0.48 — demand halves), transport hubs and Merkato dip moderately (0.81–0.88), residential zones are slightly busier (1.05–1.09) and Bole is the clear weekend winner (1.34). Bole's day-of-week shape builds from ~34 trips Mon–Thu to 41 on Friday and peaks at 49.7 on Saturday, consistent with nightlife and leisure travel.

B1.4. Demand grows steadily: weekly trips in the 11 always-open zones rose from ~43,200 to ~60,300 (+39.6%), a linear trend of ≈ +500 trips/week each week (~1% per week); mean trips per zone-hour climbs every month from 24.1 (Jan) to 34.0 (Oct). A model trained on raw history would under-forecast November, so it needs a trend feature (e.g. days_since_start) or recent-level lags; extrapolation suggests early-November weekly volume ≈ 63,700, ~5% above the October average.

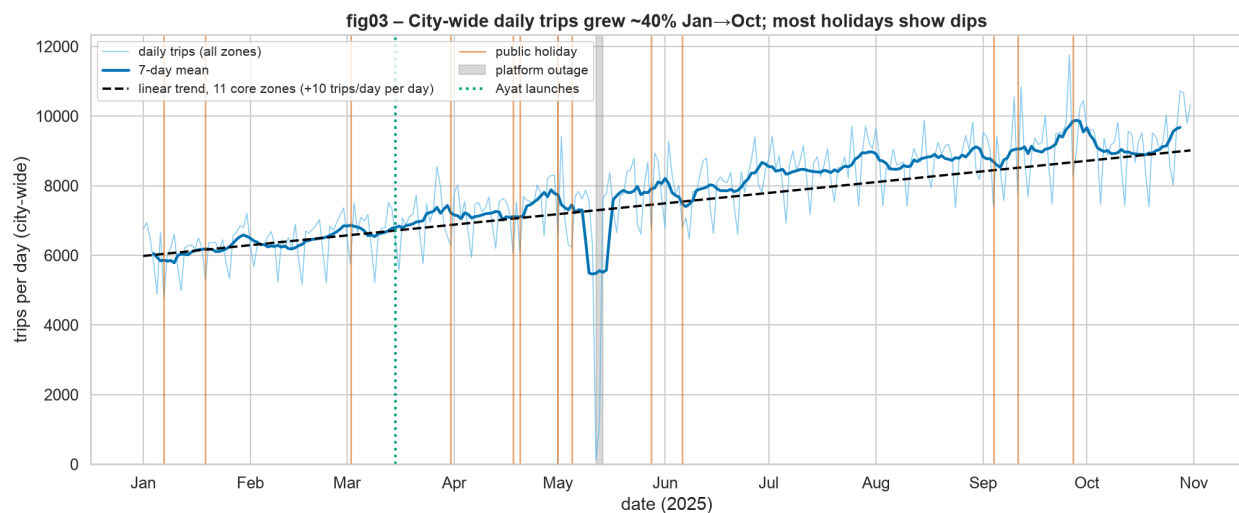


fig03_demand_trend_with_holidays.png — Daily city-wide trips Jan–Oct with 7-day mean, linear trend and public holidays. So what: demand grows ~40% over ten months, so the model needs a trend feature or it will under-forecast November.

fig04 – Weekday commuter peaks city-wide; Kazanchis empties at weekends while Bole gets busier

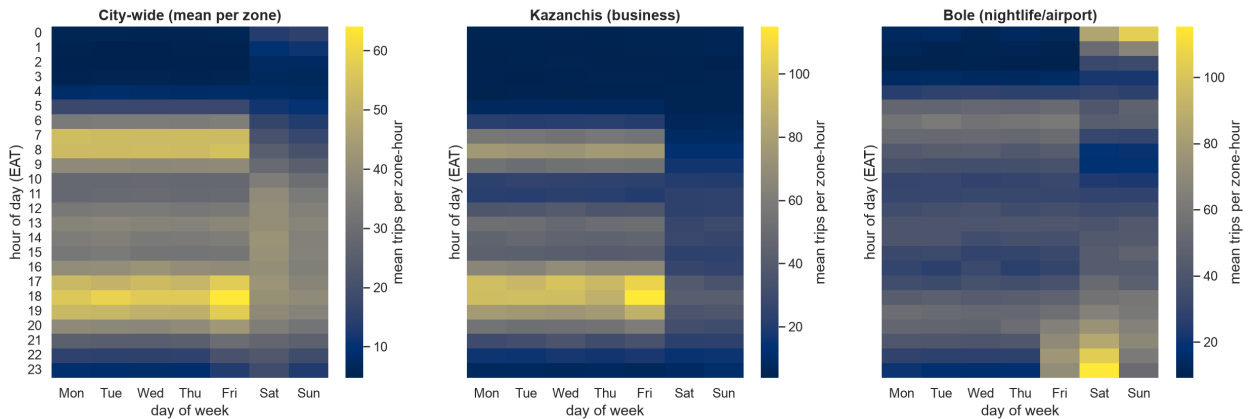


fig04_hour_by_weekday_heatmap.png — Mean trips by hour × weekday city-wide, for Kazanchis and for Bole. So what: one city-wide profile is not enough — business zones empty at weekends while Bole peaks on Friday/Saturday nights, so zone × hour × weekday interactions are essential.

fig05 – Weekday vs weekend hourly profile by zone type

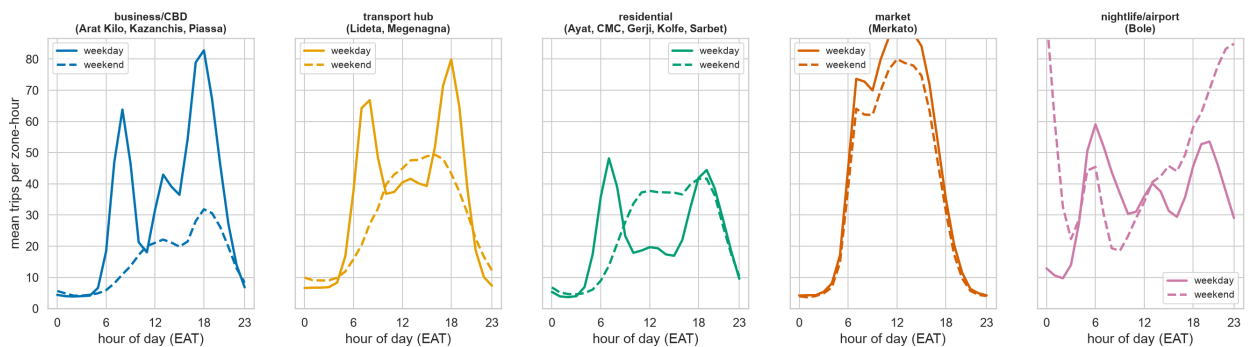


fig05_zone_profiles.png — Weekday vs weekend hourly profile for the five zone types. So what: CBD/hub zones have 08:00 and 18:00 commuter peaks, residential zones peak at 07:00/19:00, Merkato is a midday plateau and Bole has airport and night peaks.

3.2 Weather

B2.1. The weather file is on **UTC**: on the raw clock the temperature peaks at 12:00 (solar noon), after +3 h it peaks at 15:00 EAT with the minimum at 03:00 — a normal diurnal cycle. The rain–demand correlation confirms it: it is 0.42 at the corrected alignment and collapses to 0.06–0.12 for any other shift (0.056 with no correction), and the rainy-hour uplift drops from +21% to ~+6–9%. Joining the raw clock would hide most of the weather signal.

B2.2. Rain raises demand by ~27–29% in every zone type **except Merkato, where it falls by ~21%** — an open-air market loses shoppers when it rains, while elsewhere rain converts walkers and minibus riders into ride-hail customers. So rain does not increase demand everywhere, and a model needs a rain × zone interaction (trees will learn it from zone + rain_mm).

B2.3. The response is monotonic but **sub-linear**: city-wide light rain (≤ 2.5 mm/h) gives +13%, moderate +30%, heavy +46%, so each extra millimetre adds less — the effect begins to saturate in heavy rain, plausibly because driver supply caps the trips served. Merkato mirrors the pattern downward (0.87 → 0.72 → 0.57).

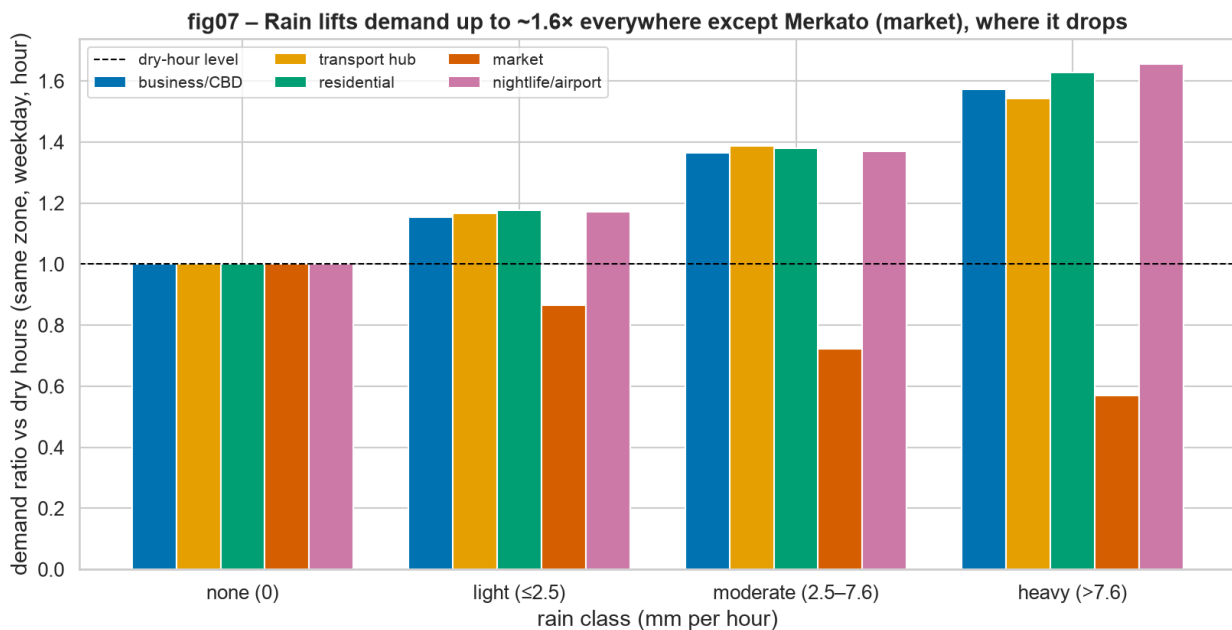


fig07_rain_effect.png — Demand ratio vs dry hours by rain class and zone type. So what: rain lifts demand roughly 1.15× (light) to 1.6× (heavy) in every zone type except Merkato, where outdoor shopping falls — the effect is sub-linear and zone-dependent.

3.3 Events and holidays

B3.1. Most holidays **reduce** city demand: Good Friday (index 0.74), Genna (0.76) and Eid al-Adha (0.79) are the biggest drops, while Sunday holidays (Timkat, Adwa) and Patriots' Victory Day are near or slightly above normal (1.01–1.08). The zone table shows the reaction is local: business zones (Arat Kilo, Kazanchis, Piassa) and Merkato fall to ~0.4–0.6 on almost every holiday, whereas Bole and the residential zones often rise (up to 1.3–1.7, e.g. 5 May) as people visit family and go out. On Meskel (27 Sep) Kazanchis and Piassa rise (1.39 / 1.18) because the Meskel Square celebrations are next to them.

B3.2. All three windows show strong uplift in the match zone vs the same ordinary zone-hours, but the **2 hours after the final whistle are largest (+124%)**, followed by the 2 hours before kick-off (+75%) and the match itself (+36%, when fans are inside the stadium). This justifies modeling separate pre/during/post flags and extending event windows past the end time.

B3.3. Ranked by whole-window effect: football (+75%), concerts (+31%, again peaking after the show at +85%) and conferences (+19%) have significant positive effects; road closures significantly **reduce** demand (–24%, –31% while active). Exhibitions (+3%) and sports runs (–12% with only 2 events, CI ±14%) have **no measurable effect**, so a separate feature for them would only add noise.

B3.4. (a) Five of the six cancelled events leave no meaningful footprint (–9% to +17%, within normal hour-to-hour noise), confirming they should be excluded. The exception, the Kazanchis football match of 22 Feb (+75%), behaves like a match that actually happened — likely a mislabelled status or a rescheduled game, worth flagging. (b) Large unexplained spikes: **Kazanchis 26 Sep (×1.74)** = Meskel eve (Demera bonfire at Meskel Square), **Merkato 19 Apr (×1.47)** = Saturday before Fasika (Easter shopping), **Merkato 10 Sep (×1.36)** = eve of Enkutatash (New Year shopping). The Ayat spikes in April–June are early post-launch volatility. Holiday eves are missing from the calendar and could be added as a feature.

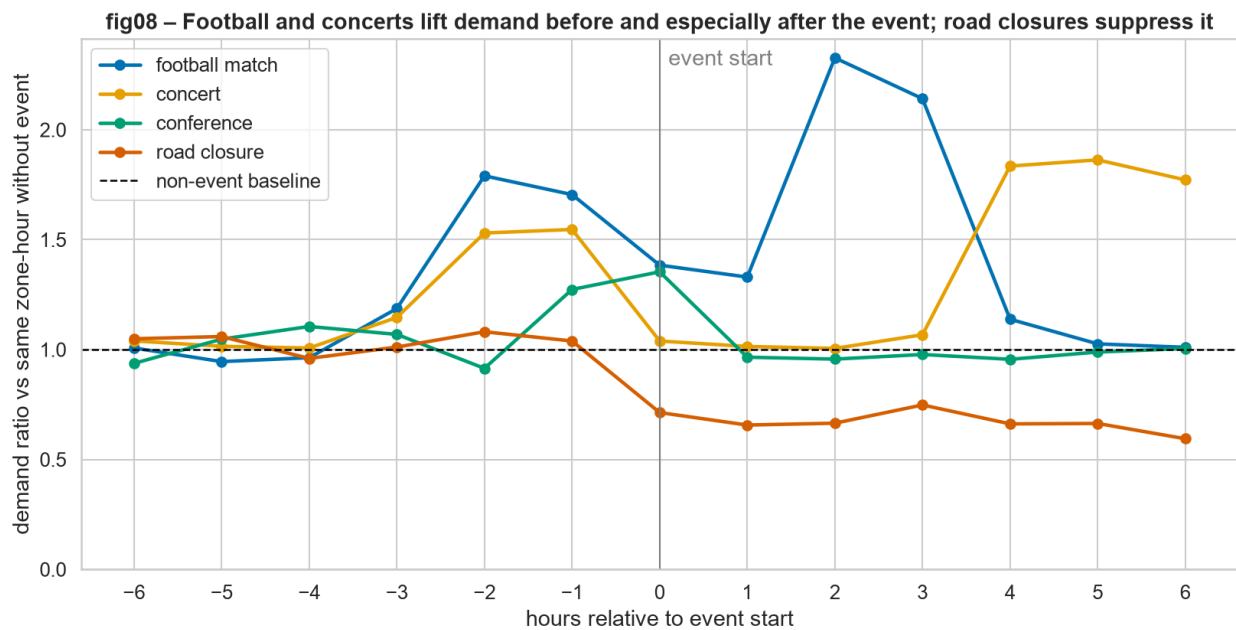


fig08_event_study.png — Demand ratio from –6 h to +6 h around event start for four event types vs the non-event baseline. So what: football and concerts raise demand before and especially after the event, conferences slightly, while road closures suppress it — event windows must extend past the end time.

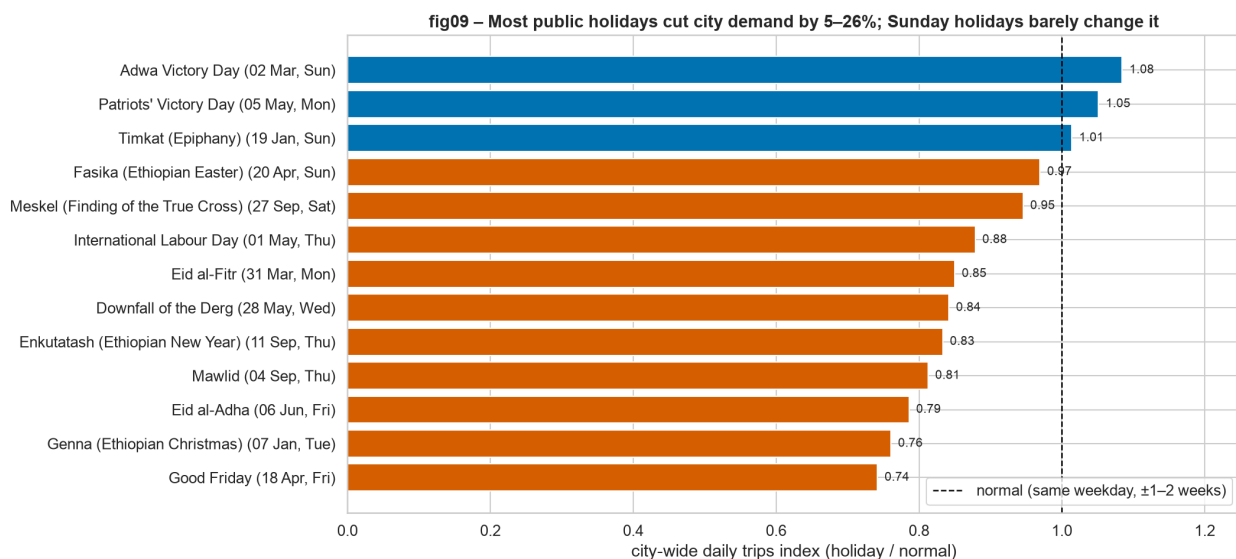


fig09_holiday_effects.png — City-wide daily trips on each public holiday relative to the same weekday 1–2 weeks around it. So what: most holidays cut demand by 5–26% (Good Friday, Genna, Eid al-Adha most), while Sunday holidays are near normal — a single holiday flag captures most of the effect.

3.4 Data quality and operational columns

B4.1. Trips correlate almost perfectly with active drivers ($r = 0.99$, ~1.31 trips per driver-hour), moderately with wait time (0.53) and not at all with fare (0.01). These are **consequences** of demand: drivers log on and waits lengthen *because* riders are requesting, and fares are set per zone. None of them is known for next Tuesday at 18:00, so using them as inputs would leak the target; they are excluded from the model (D4) but used for the demo's driver and revenue estimates.

B4.2. Of 87,552 possible zone-hours, 82,979 are usable. There are three different kinds of gap: (1) one **platform outage** from 12 May 02:00 to 13 May 19:00 (42 h, 504 zone-hours) where no zone has any row — excluded, not imputed; (2) **Ayat not yet launched** before 15 Mar (1,752 zone-hours) — excluded, they are not zero-demand hours; (3) **2,317 random missing or invalid records** (dropped rows, –1 sentinels,

impossible spikes) spread evenly across zones (155–211 each) — excluded from the target and handled by lag features' NaN tolerance.

B4.3. After removing each zone's weekly level and its weekday-hour shape, demand on the 1st–3rd is +1.4% ($\pm 0.7\%$) and on the 25th–end +0.7% ($\pm 0.5\%$) vs ordinary days; the day-of-month curve wobbles by ± 2 –3% with no clear payday bump. The effect is statistically detectable but tiny compared with weather (+13–46%) or events (+75%), so `is_payday_window` is kept only as a cheap optional feature and is expected to add little.

4. Visualisation pack (Deliverable C)

Twelve figures were produced at 150 dpi with a colour-blind-safe (Okabe–Ito) palette. Each has a caption stating what it shows and the *so what* for the forecast. Figures 1–9 appear next to the sections they support. Figures 10–12 summarise the model and appear in Section 5. All captions are listed here for reference:

Figure	Caption
fig01_gaps_and_missingness.png	Missing/invalid values per column in the three raw tables and a zone × day map of missing trip hours. So what: the gaps are structured — Ayat did not exist before 15 Mar and a 42-hour city-wide outage hit 12–13 May — so they are excluded rather than imputed as zero demand.
fig02_before_after_cleaning.png	Raw vs cleaned distributions of temperature, rain readings and trips. So what: a July block recorded in °F (60–77), –9999 rain sentinels and trip spikes far beyond driver capacity would each have distorted the model; cleaning removes them without changing the normal range.
fig03_demand_trend_with_holidays.png	Daily city-wide trips Jan–Oct with 7-day mean, linear trend and public holidays. So what: demand grows ~40% over ten months, so the model needs a trend feature or it will under-forecast November.
fig04_hour_by_weekday_heatmap.png	Mean trips by hour × weekday city-wide, for Kazanchis and for Bole. So what: one city-wide profile is not enough — business zones empty at weekends while Bole peaks on Friday/Saturday nights, so zone × hour × weekday interactions are essential.
fig05_zone_profiles.png	Weekday vs weekend hourly profile for the five zone types. So what: CBD/hub zones have 08:00 and 18:00 commuter peaks, residential zones peak at 07:00/19:00, Merkato is a midday plateau and Bole has airport and night peaks.
fig06_weather_timezone_check.png	Daily temperature curve on the raw vs corrected clock, and rain–demand correlation for each clock shift. So what: the weather file is in UTC; the +3 h correction puts the temperature peak at 15:00 and raises the rain–demand correlation from 0.06 to 0.42.
fig07_rain_effect.png	Demand ratio vs dry hours by rain class and zone type. So what: rain lifts demand roughly 1.15× (light) to 1.6× (heavy) in every zone type except Merkato, where outdoor shopping falls — the effect is sub-linear and zone-dependent.
fig08_event_study.png	Demand ratio from –6 h to +6 h around event start for four event types vs the non-event baseline. So what: football and concerts raise demand before and especially after the event, conferences slightly, while road closures suppress it — event windows must extend past the end time.
fig09_holiday_effects.png	City-wide daily trips on each public holiday relative to the same weekday 1–2 weeks around it. So what: most holidays cut demand by 5–26% (Good Friday, Genna, Eid al-Adha most), while Sunday holidays are near normal — a single holiday flag captures most of the effect.
fig10_model_comparison.png	Validation RMSE of every model with both baselines marked and rolling-origin error bars. So what: the optimised LightGBM (Poisson loss, 2–5-week lags, 8-week zone×weekday×hour profile) reaches RMSE 8.47 vs 11.30 for seasonal-naive (–25%) and 27.5 for the mean, and stays ahead on every rolling-origin fold (9.21 ± 0.71).
fig11_forecast_vs_actual.png	Forecast vs actual hourly trips for three contrasting zones over the validation fortnight. So what: the model follows daily peaks and weekend shifts closely in all three zones (MAE 5.85 trips ≈ 4.5 drivers per zone-hour); the largest misses Merkato on the tallest event and rain spikes, which it under-shoots.
fig12_feature_importance.png	Permutation importance of the top features, with weather and event features highlighted. So what: the 8-week zone×weekday×hour profile dominates (it encodes the zone's normal hour), followed by the multi-week lag mean; rain is the strongest non-history feature, confirming the corrected weather join adds real signal, while individual event flags matter only in the few hours they cover.

5. Modelling & evaluation (Deliverable D)

5.1 Rules followed

- **Forecast-time features only (Rule 6):** November is forecast up to 14 days ahead, so trip history is used only at lags of 14 days (336 h) or more. Weather comes from the provided forecast rows and events from the scheduled calendar.
- **Chronological validation (Rule 7):** the main split trains before 18 Oct and validates on 18–31 Oct. Four rolling-origin folds start on 6 Sep, 20 Sep, 4 Oct and 18 Oct, each 14 days long. Nothing is shuffled.
- **Weather and events used (Rule 5), and nothing fitted on the test file (Rule 8).**

Leakage audit (D4). Every model feature is known at forecast time: calendar fields, trip history at least 14 days old (2–5 weeks back), the weather *forecast* and the scheduled event calendar. Excluded: active_drivers, avg_wait_min, avg_fare_birr (consequences of demand that only exist in history), 1 h/24 h lags (unknown 2–14 days ahead) and month (November is unseen). Adding active_drivers cut validation RMSE (for the model at that stage) from 9.06 to **4.25** — a fake 53% improvement, because the driver count is effectively a measurement of the demand being predicted; it does not exist for November, so the model could not even run in production.

5.2 Baselines and model comparison (D1, D2)

Model	RMSE	MAE	Type
Mean predictor	27.54	20.69	baseline
Seasonal-naïve (all history)	11.30	7.36	baseline
Seasonal-naïve (last 4 weeks)	9.94	6.64	baseline
Ridge regression	11.29	7.55	model
Random forest	10.31	6.76	model
LightGBM (raw target)	9.09	6.08	model
LightGBM (level-normalised target)	9.21	6.11	model
LightGBM optimised (Poisson + long lags + profile)	8.47	5.85	model

Baselines. The mean predictor is useless (RMSE 27.5). The seasonal-naïve baseline (mean of the same zone × weekday × hour) gets RMSE 11.3 / MAE 7.4 using all history, and **9.9 / 6.6 using only the last 4 weeks** — the recent version wins because demand grew ~40% over the year, so old weeks drag the average down. Any real model must beat 9.94.

Models. LightGBM with the raw target wins (RMSE 9.09, MAE 6.08, 2 s to train), ahead of the level-normalised LightGBM (9.21), random forest (10.31) and ridge (11.30). Gradient boosting handles the zone × hour × weekday interactions and the non-linear rain/event effects natively, and the 14-day lags give it the recent level so it copes with the trend; ridge cannot model interactions without hand-built terms. The winner beats the strongest baseline by ~9% RMSE.

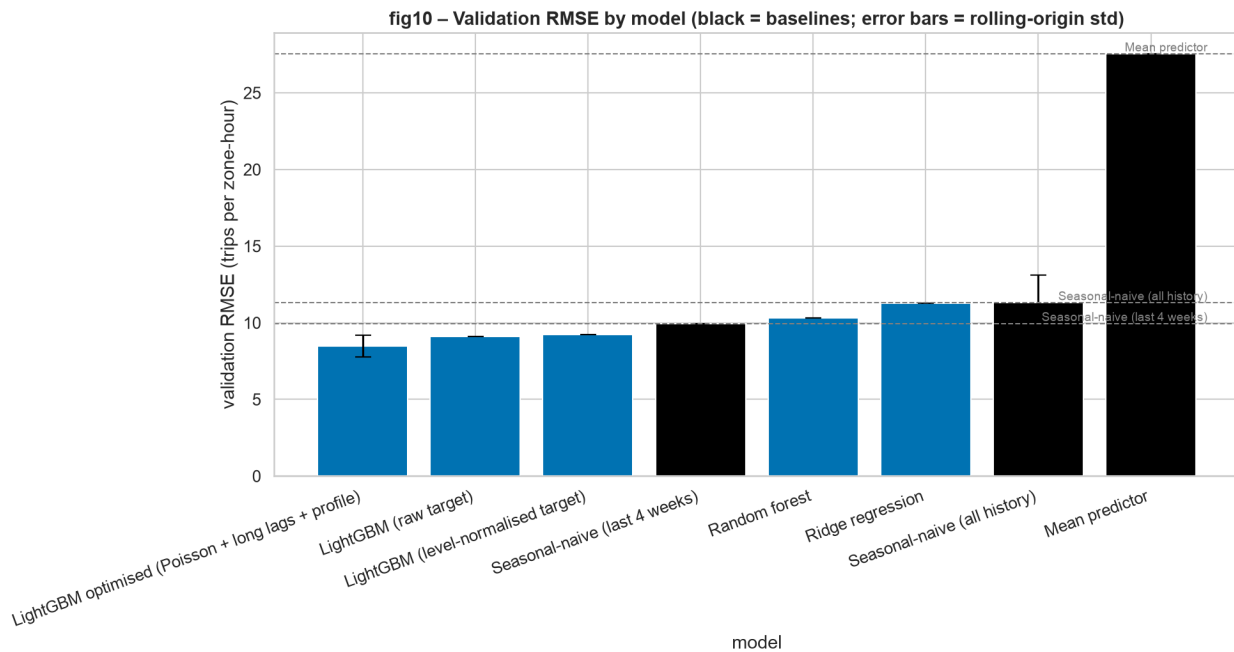


fig10_model_comparison.png — Validation RMSE of every model with both baselines marked and rolling-origin error bars. So what: the optimised LightGBM (Poisson loss, 2–5-week lags, 8-week zone×weekday×hour profile) reaches RMSE 8.47 vs 11.30 for seasonal-naive (–25%) and 27.5 for the mean, and stays ahead on every rolling-origin fold (9.21 ± 0.71).

5.3 Tuning, rolling-origin validation and ablation (D6, D3, D5)

Note: sections 5.3–5.4 describe the model **before** the optimisation round in 5.5, so their scores (e.g. rolling RMSE 9.67, main RMSE 9.06) are higher than the final model's.

Tuning. A 20-trial random search over 6 LightGBM hyperparameters, scored by mean RMSE on the three earlier rolling folds (the main fortnight was not used for tuning), picked `num_leaves` 63, `learning_rate` 0.05, 1000 trees, `min_child_samples` 20, `colsample` 0.6, `reg_lambda` 5. Fold CV RMSE improved from 9.99 to 9.87, and the main validation RMSE from 9.088 to 9.058 — a small gain, as is typical once features are good.

Rolling origin. Across 4 folds the final model scores **RMSE 9.67 ± 0.83** vs **12.32 ± 1.83** for seasonal-naive (all history): it is better in every fold and roughly half as variable. The worst fold is 20 Sep – 3 Oct (10.81), which contains Meskel (27 Sep) and its eve — a holiday whose effect is local (Kazanchis/Piassa up, business zones down) and seen only once in training, plus the unlisted Meskel-eve spike found in B3.4. The two October folds are the easiest (~9.05).

Ablation. Calendar + zone + trend/lag features alone give RMSE 9.74. Adding weather cuts it to 9.21 (–5.4%), events alone to 9.64 (–1.1%), and both together to **9.06 (–7.0%)** — the gains are roughly additive. Both joins were worth it: weather is the bigger win because rain hits every zone for many hours, while events affect few zone-hours (but those hours have large errors, see D7).

5.4 Error analysis and response (D7, D8)

Where the model fails. (a) Absolute error is largest in the busiest zones (Merkato, Kazanchis, Bole, Megenagna: RMSE 11–13), but relative error is flat at 17–20% of mean in every zone, including the newest (Ayat 20%). By hour, absolute error peaks at the 07:00 and 17:00–18:00 rush (RMSE 12–15), relative error is highest at night (~30%, small counts). Weekends are slightly worse than weekdays (20.5% vs 17.4%), and the model has a small under-forecast bias (–1 trip). (b) The 10 largest errors fall in three groups: **Kazanchis football days (18 & 25 Oct)** over-predicted by 50–70 trips — the model applies the average match uplift, but these matches drew far less (possibly low-profile fixtures; attendance alone doesn't capture it); **Bole events (exhibition night 18 Oct, two concerts 26 Oct)** under-predicted by ~55–60 — overlapping/late-night events produce bigger surges than the average; and **unlisted spikes** — Megenagna evening rush on 28–29 Oct in

rain (transport-hub surge larger than the average rain effect) and Merkato on Saturday 18 Oct morning/midday (no listed event; likely a market or religious-day crowd).

Response. Following B3.4 and D7, we added two forecast-time features: `is_holiday_eve` (the day before a public holiday) and `post_event_attendance` (attendance $\times$ post-event window, so big crowds leaving produce bigger surges). They improved the rolling-origin mean RMSE from 9.67 to **9.58** and the main fortnight from 9.06 to 9.03, so they were kept. The gain is modest: the largest D7 errors come from events that differ from their type's average and from unlisted events, which no calendar feature can fix.

5.5 Optimisation round (D10)

A further round of experiments was run on all four rolling folds. Each change was kept only if it lowered the mean rolling RMSE:

Change	Main RMSE	Rolling mean RMSE
Tuned model after D8 (starting point)	9.03	9.58
+ lags of 4 and 5 weeks, their mean, median and spread, lag ratio, hour $\times$ weekday	8.87	9.60
+ 8-week zone $\times$ weekday $\times$ hour profile (refit inside each fold)	8.70	9.44
+ Poisson objective (count data)	8.57	9.33
+ smaller trees (15 leaves, 1,500 rounds), 3-seed average $\rightarrow$ final	8.47	9.21

Tried and rejected: weighting recent data more heavily, larger trees, a Tweedie objective, and dropping the weak event features.

Result. The optimised model lowers main-validation RMSE from 9.03 to 8.47 (–6%) and mean rolling-origin RMSE from 9.58 to 9.21, beating the previous final on the folds, so it is adopted. Most of the gain comes from the 8-week zone $\times$ weekday $\times$ hour profile (a robust estimate of the 'normal' hour that adapts to recent growth) and the Poisson loss, which matches count data whose noise grows with demand. The extra 4–5-week lags smooth out one-off spikes in the 2–3-week lags. Seed averaging and smaller trees cut variance. Fold spread rises slightly (0.62 $\rightarrow$ 0.71), so the gain is real but not uniform across fortnights.

5.6 What the error means in practice (D9)

On the validation fortnight the final model has **RMSE 8.47** and **MAE 5.85** trips per zone-hour, against an average demand of about 33 trips. A typical miss is 17.5% of demand. At ≈ 1.31 trips per driver-hour, the driver plan is typically off by about **4–5 drivers per zone per hour**, and by about 6–7 in the hardest hours (morning and evening rush, event exits). That is 25% better than the seasonal-naïve baseline and 69% better than predicting the mean.

fig11 – Forecast vs actual hourly trips, validation fortnight

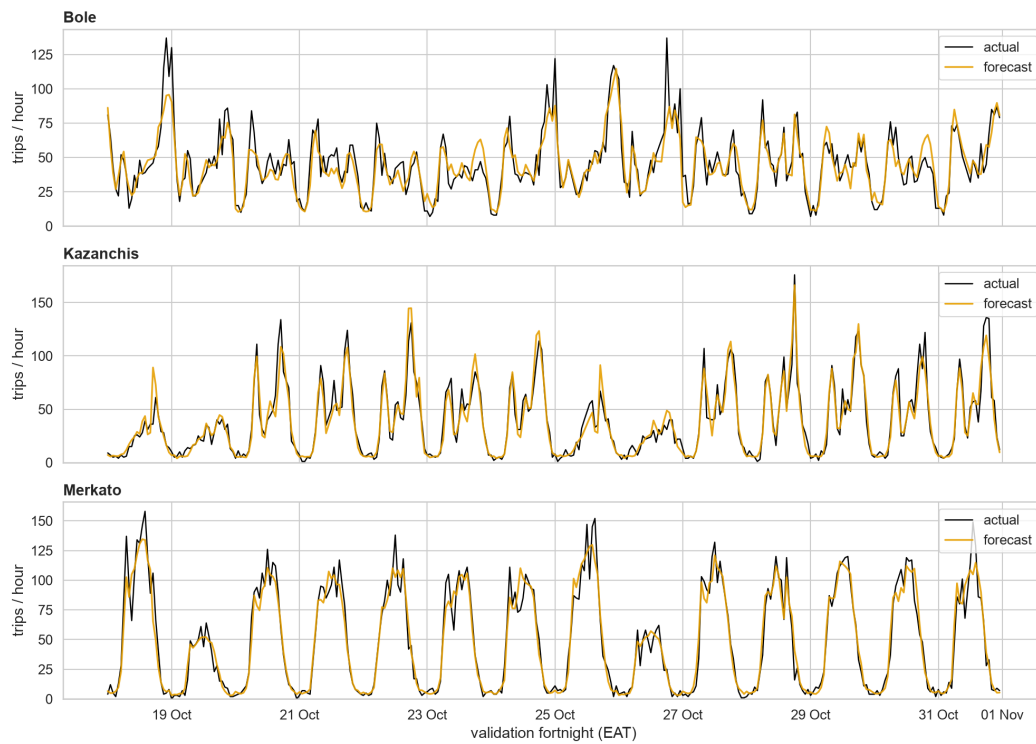


fig11_forecast_vs_actual.png — Forecast vs actual hourly trips for three contrasting zones over the validation fortnight. So what: the model follows daily peaks and weekend shifts closely in all three zones (MAE 5.85 trips $\approx$ 4.5 drivers per zone-hour); the largest misses are on the tallest event and rain spikes, which it under-shoots.

5.7 Feature importance

Permutation importance on the validation fortnight: how much RMSE rises when one feature is shuffled.

Feature	RMSE increase	Group
prof8	25.349	lag
wk_mean	0.505	lag
zone	0.488	calendar
rain_mm	0.378	weather
hour_dow	0.193	lag
day_of_week	0.150	calendar
roll_mean_168h_lag336	0.124	lag
wk_med	0.122	lag
hour	0.094	calendar
day_of_month	0.074	calendar
hour_sin	0.073	calendar
is_payday_window	0.065	calendar

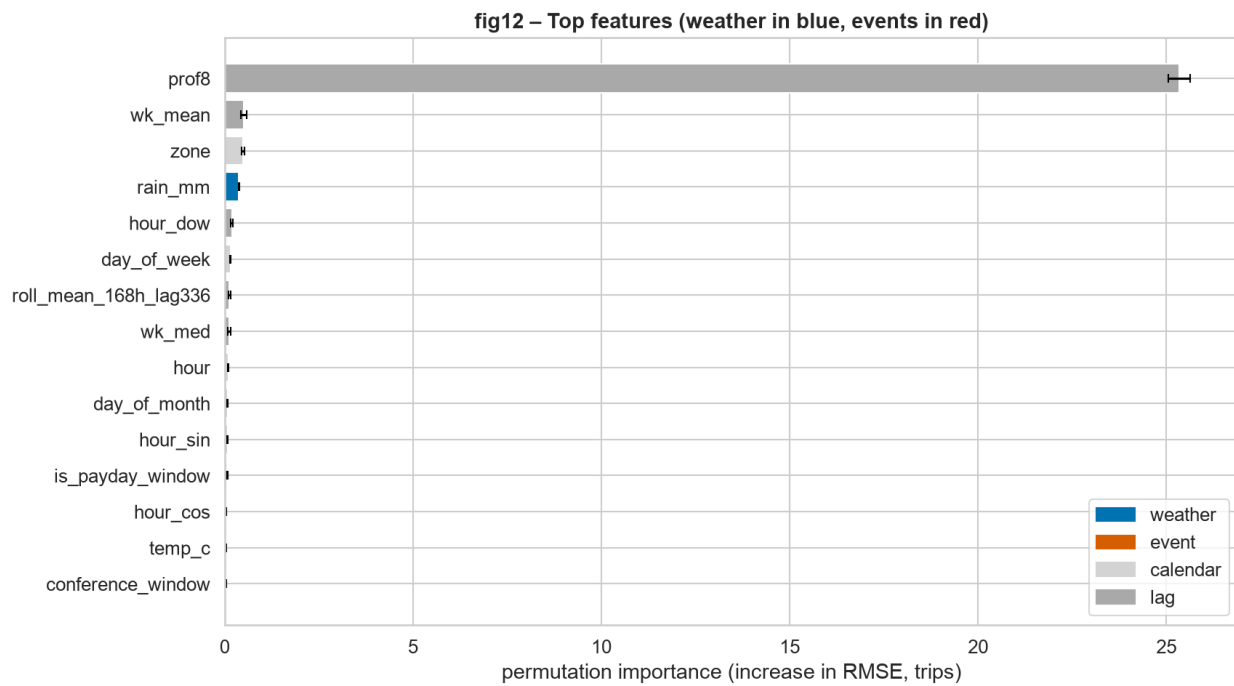


fig12_feature_importance.png — Permutation importance of the top features, with weather and event features highlighted. So what: the 8-week zone×weekday×hour profile dominates (it encodes the zone's normal hour), followed by the multi-week lag mean; rain is the strongest non-history feature, confirming the corrected weather join adds real signal, while individual event flags matter only in the few hours they cover.

5.8 Final model and submission

- Retrained on all history (1 Jan – 31 Oct) and saved to `models/final_model.joblib`. The file holds the 3 models, feature list, 8-week profile, parameters and trips-per-driver ratio.
- `submission/team_alpha_submission.csv`: 4,032 rows with columns `row_id` and `predicted_trips`, in template order, with no blank or negative values.
- Mean predicted November demand is 32.8 trips per zone-hour, against an October average of 32.9 — consistent with the trend.

6. Forecast demo app (Deliverable E)

app/app.py is a Streamlit app for an operations manager. The **only inputs are a zone and a date** (1–14 Nov 2025). Weather and events are looked up automatically from the cleaned tables bundled in app/assets/, which app/build_assets.py creates from the processed data and the final model.

Output	How it is computed
24-hour forecast (table and curve)	final model applied to that zone-day's forecast-time features
Comparison with a typical day	zone × weekday × hour mean over the last 8 weeks, drawn as a dashed line
Peak hour	hour with the highest forecast
Drivers needed per hour	forecast ÷ 1.31 trips per driver-hour, rounded up
Expected gross fares	forecast × the zone's historical average fare
What was looked up	e.g. 'rain in 4 hours, heaviest 13.0 mm at 17:00; Premier league match at Addis Ababa Stadium 19:00–21:00, ~32,000 people'
Chart shading	event windows (orange) and rain hours (blue)
Dates outside 1–14 Nov	friendly warning, no crash
Download	hourly table as CSV

```
source .venv/bin/activate
streamlit run app/app.py      # opens http://localhost:8501
```

A test harness checked several zone and date combinations (a football day in Kazanchis, a concert day in Bole, a rainy day in Piassa, an out-of-range date). The app's numbers equal the submission file exactly.

7. Reproducibility & project structure (Deliverable G)

7.1 Environment

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

requirements.txt pins Python 3.13 packages: pandas 3.0.3, numpy 2.4.6, pyarrow 25.0.1, scikit-learn 1.9.0, lightgbm 4.7.0, matplotlib 3.11.0, seaborn 0.13.2, streamlit 1.65.0, python-pptx, jupyter and nbconvert. All paths are relative and random seeds are fixed (42 for experiments, 1–3 for the three final training runs).

Common pitfall: another Python installation (e.g. base Anaconda) may be missing lightgbm or pyarrow; the saved model cannot be loaded without pyarrow. Always activate .venv first, and pick it as the notebook kernel.

7.2 Run order

```
jupyter nbconvert --to notebook --execute --inplace notebooks/01_cleaning_and_integration.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/02_analysis_report.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/04_modeling_and_evaluation.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/03_visualizations.ipynb # after 04
python app/build_assets.py
```

7.3 Reusable pipeline (src/)

The notebook logic is also packaged as importable modules. src.cleaning was checked to write exactly the same master tables, weather, events and cleaning log as notebook 01, and src.predict reproduces the submission file exactly (maximum difference 0.0).

Module	Responsibility
src/config.py	paths, seed, date ranges, zones, validation cut-offs
src/cleaning.py	timestamp parsing, zone/event normalisation, trip/weather/event cleaning, interval join, master tables, 13 integrity checks
src/features.py	model feature list, long lags, holiday-eve, 8-week profile, leakage guard
src/train.py	fit / predict, rolling-origin evaluate, train_final
src/predict.py	make_submission, forecast_zone_day

```
python -m src.cleaning # raw -> data/processed/
python -m src.train    # validation + models/final_model.joblib
python -m src.predict  # submission CSV
python -m src.predict --zone Bole --date 2025-11-05 # one zone-day
```

7.4 Folder layout

```
team_alpha/
|-- README.md, requirements.txt
|-- submission/team_alpha_submission.csv
|-- data/raw/                original 5 CSVs (never edited)
|-- data/processed/         master_train, master_test, data_dictionary_master, cleaned weather/events
|-- notebooks/              01 cleaning, 02 analysis, 03 visualisations, 04 modelling
|-- figures/                fig01 ... fig12, figure_captions.md
|-- reports/                A_, B_, D_ HTML reports, cleaning log, model results, importance
|-- models/final_model.joblib
|-- src/                    reusable pipeline
|-- app/                    app.py, build_assets.py, requirements.txt, assets/
|-- docs/                   this document and its generator
|-- presentation/           team_alpha_slides
```

8. Limitations and next steps

- **Rare, one-off days** (e.g. Meskel and its eve, which appear once in training) remain the biggest source of error. A holiday-specific prior or zone × holiday-type effects would help.
- **Event peaks are under-predicted.** The tallest post-match surges exceed what the 48 football matches in the history can teach. An event-study multiplier applied on top of the model, or quantile forecasts for staffing, could close the gap.
- **Weather forecast error:** the model is validated on observed weather, but November uses forecast weather, which is noisier. Training with noise added to the weather features would make it more robust.
- **Driver supply** caps observed trips in heavy rain, so the model learns *served* demand, not *requested* demand. Modelling unmet demand would need request-level data.
- **Operational:** retrain weekly as new data arrive, track live error by zone, and host the app (Streamlit Community Cloud) so managers can use it without a laptop.